Installation Guide

Project: Enterprise Quantum Computing Platform — QFT Bank **Student:** Syed Muhammad Tahoor Ali **Diploma:** EduQual Level 6, Diploma in Artificial Intelligence Operations (ANPP-OP)

1. Purpose

This guide covers the prerequisites and one-time setup needed to obtain the source and run the platform locally. Running the platform — via Docker Compose, Kubernetes, or Terraform — is covered separately in the Deployment Guide. This document gets a fresh machine to the point where any of those paths will work.

2. Prerequisites

The platform was developed and verified on Kali Linux with Python 3.13. The following are required for the full local experience; not every path needs every tool (see the table).

Tool	Version used	Needed for
Git	any recent	Cloning the repository
Python	3.13	Backend (local dev, tests)
Node.js	22 (Alpine in Docker)	Frontend build
Docker	recent	Docker Compose path; building images
Docker Compose	v2 (`docker compose`)	Compose path
Minikube	recent	Kubernetes path
kubectl	matching cluster	Kubernetes path
Terraform	recent	Infrastructure-as-Code path

For a minimal "just run the tests and the backend" setup, only Git and Python are strictly required, because persistence falls back to in-memory when no database is present.

3. Obtaining the Source

```
git clone https://github.com/tahoorshah/enterprise-quantum-computing-platform.git
cd enterprise-quantum-computing-platform
```

The repository is organised into `backend/` (FastAPI application and tests), `frontend/` (React SPA and nginx config), `infra/` (Kubernetes manifests and Terraform), `diagrams/` (the 15 architecture diagrams), and `docs/` (this documentation set).

4. Environment Configuration

The Docker Compose path reads configuration from a `.env` file. A template is provided:

```
cp .env.example .env
```

The template contains development-only values:

Variable	Default	Meaning
`POSTGRES_USER`	`qft_admin`	Database user
`POSTGRES_PASSWORD`	`changeme_local_dev_only`	Database password (development only)
`POSTGRES_DB`	`qft_quantum_db`	Database name
`POSTGRES_PORT`	`5432`	Host port for PostgreSQL
`REDIS_PORT`	`6379`	Host port for Redis
`BACKEND_PORT`	`8000`	Host port for the backend
`DATABASE_URL`	see template	Full connection string used by the backend
`REDIS_URL`	see template	Redis connection string

The password is deliberately a placeholder. It is safe for local assessment because nothing is exposed outside the machine, but it would

be replaced by a secret manager in production.

---

5. Backend Setup (Local Development)

For running the backend directly (without containers) or running the test suite:

```
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

The `requirements.txt` pins exact versions verified to work together, including the quantum frameworks (`qiskit==2.5.0`, `qiskit-aer==0.17.2`, `pennylane==0.45.1`, `cirq==1.7.0`), the scientific stack (`numpy`, `scipy`, `pandas`, `scikit-learn`), FastAPI/Uvicorn, and the visualisation libraries (`matplotlib`, `plotly`).

5.1 Quantum sanity check

Before trusting any results, verify the simulator produces genuine quantum behaviour:

```
python app/quantum/sanity_check.py
```

This constructs a known circuit (a Bell state) and confirms the measured distribution is a true ~50/50 split, proving the simulation is real rather than fabricated.

5.2 Running the test suite

```
python3 -m pytest -q
```

The suite covers all modules. A single deprecation warning from FastAPI's test client is expected and does not affect correctness.

---

6. Frontend Setup (Local Development)

```
cd frontend
npm ci
```

`npm ci` installs exactly the locked dependency versions from `package-lock.json`. The frontend targets React 19 built with Vite.

The API base URL is environment-aware. In local development, leaving `VITE_API_BASE` unset points the app at `http://localhost:8000` (a directly-run Uvicorn backend). In the production build it is set to an empty string so the app uses relative `/api` paths, which nginx proxies. This lets one codebase serve both environments.

---

7. Verifying the Installation

With the backend running (see the Deployment Guide for the various ways to start it), confirm health:

```
curl http://localhost:8000/health
```

A healthy response looks like `{"status":"healthy","storage_backend":"postgresql"}` when a database is connected, or `{"status":"healthy","storage_backend":"in-memory"}` when running without one. Either is a valid, working state.

Interactive API documentation is then available at `http://localhost:8000/docs`.

---

8. Next Steps

With prerequisites installed and the source obtained, proceed to the Deployment Guide to run the platform via Docker Compose (simplest), Kubernetes (the assessed deployment target), or Terraform (Infrastructure-as-Code).